

## 資料結構 第三週 二元搜尋樹

本週介紹二元搜尋樹（Binary Search Tree，簡稱 BST）。每個節點的左子樹只放比它小的鍵，右子樹只放比它大的鍵。

在平衡的情況下，搜尋、插入與刪除都只需要  $O(\log n)$  的時間；如果依序插入已排序的資料，樹會退化成一條鏈，最壞情況變成  $O(n)$ 。

課堂範例：依序插入 50、30、70、20、40、60、80，畫出完成後的樹，並寫出「中序走訪」的結果。

Café résumé naïve — “quoted” text & symbols:  $100\% \leq 200 \cdot \frac{1}{4} \text{ cup}$ .

## 第二節 插入與刪除

插入：從根開始比較，小於往左、大於往右，直到遇到空位為止。

刪除分三種情況：一、沒有子節點，直接移除；二、只有一個子節點，用子節點取代；三、有兩個子節點，找右子樹的最小值（中序後繼者）來替換。

罕用字測試：尗（U+2000B）與「覷」。

### 第三節 走訪與應用

前序、中序、後序三種走訪方式都可以用遞迴寫出。中序走訪二元搜尋樹會得到由小到大排好的序列。

應用：字典、資料庫索引、排程器。 *Self-balancing* trees such as AVL and red-black trees keep the height at  $O(\log n)$ .

#### 第四節 作業與參考資料

作業三：實作 insert、delete、inorder 三個函式，下週二上課前繳交。

參考資料：Introduction to Algorithms 第十二章。